



Helmut

INTERNER BETRIEBSBERICHT · DATENMOTOR

Helmut · Nohut

VERTRAULICH

Inhalt

TEIL I · AUDITBERICHT

0. Kurzfassung (Abschlussurteil)	
1. Geprüfter Stand (Phase 1 — Audit-Snapshot)	Belastbare technische Wahrheit des Helmut-Datenmotors — geprüft gegen echten Code, Production-Datenbank und Deployment-Stand. Jede Aussage ist belegt.
2. Geprüfte / nicht prüfbare Bereiche	
3. Architektur-Überblick: das duale Datenmodell	
4. Vollständiger Prozesskatalog (Phase 3 — Steckbriefe)	GEPRÜFTER COMMIT
5. Zeitsteuerung (Phase 4)	427295c · main = Production
6. Vollständiger Datenfluss (Phase 2) — Weg eines Dokuments	KI-MODELL
7. Dokumentlebenszyklus (Phase 5)	Azure OpenAI gpt-5-mini
8. Laufzeitwerte (Phase 3) — vorhanden vs. fehlend	
9. Fehler & Überwachung (Phase 6)	
10. Datenqualität & Nachvollziehbarkeit (Phase 7)	
11. Ungenutzte Dateien / toter Code (Phase 8)	
12. Landtagsfähigkeit (Phase 9) — Berlin/Brandenburg	
13. Risiken (priorisiert)	
14. Offene Gründerentscheidungen	

TEIL II · UMSETZUNGSPLAN

P0 — Kritisch (Beobachtbarkeit & Sicherheit)
P1 — Vor Bundestagspilot (Ehrlichkeit & Datenqualität)
P2 — Vor Landtagspilot (Berlin/Brandenburg)
P3 — Später (Hygiene & Skalierung)
Zuordnung: Prozesse ohne ausreichende Laufzeitmessung → wo Messung einbauen
Offene Gründerentscheidungen

TEIL I

Auditbericht

Geprüfter Stand, vollständiger Prozesskatalog, Datenfluss, Dokumentlebenszyklus, gemessene vs. fehlende Laufzeiten, Fehlerfälle, Datenqualität, toter Code, Landtagsfähigkeit und Risiken.

Erstellt: 2026-07-16 · **Branch:** `claude/helmut-engine-audit-ifkv6r` · **Modell:** Opus **Methode:** Prüfung gegen echten Code (HEAD = Production), gegen die Production-Datenbank (Supabase `ddckuvvpcytqbyfmbvie`), gegen Vercel-Deployments und gegen die Git-/PR-Historie. Jede Aussage ist mit Datei:Zeile, DB-Abfrage oder Deployment-Beleg unterlegt. Laufzeiten sind **gemessen, nicht geschätzt**; wo keine Messung existiert, steht das ausdrücklich. Es wurde **nichts** an Production, Migrationen, Zeitplänen, Secrets oder Deployments geändert.

Wichtiger Lesehinweis zur Belastbarkeit: Dieser Bericht ersetzt ältere Berichte (`docs/AUDIT_DATENMOTOR_2026-07.md` ist nachweislich veraltet, siehe §11). Er wurde nicht aus Dokumentation abgeleitet, sondern aus Code + Live-Daten.

0. Kurzfassung (Abschlussurteil)

- Was geprüft wurde:** Kompletter Datenmotor — Crawling, Speicherung, Textverständnis (KI), Klassifikation, Matching/Relevanz, Entscheidungen, Briefing/Lage/Radar-Ausgabe, alle geplanten Prozesse (9 Vercel-Crons + 1 GitHub-Action), Fehlerbehandlung, Datenqualität, toter Code, Landtagsfähigkeit, Env/Flags/Secrets. Belege aus Code **und** Production-DB.
- Was nachgewiesen funktioniert:** Der Crawl läuft stabil (145 Quellen, 100 % Quellen-Erfolg über 20 protokollierte Läufe, 0 Laufzeitfehler); KI-Verständnis produziert Wissensobjekte (Azure `gpt-5-mini`, Ø 7,9 s/Call, 236/256 erfolgreich); das Briefing wird bei jedem Aufruf deterministisch (0 KI) aus verstandenen Wissensobjekten gebaut und funktioniert für alle drei Testprofile; Quellen-Cutover auf die relationale Architektur ist live; Kostenkontrolle (Tagesdeckel, atomare Reservierung) ist aktiv; KI-Gesamtkosten seit 01.07. ≈ **\$0,92**.
- Was NICHT nachgewiesen ist: Keine einzige persistierte Laufzeit** für Crawl, Pipeline, Lage-Check, Understanding-Batch, Briefing- oder Radar-Aufbau (nur ephemere Vercel-Logs). Der tatsächliche Prod-Wert zentraler Env-Variablen (`HELMUT_MAX_LLM_CALLS_PER_DAY`, `HELMUT_V3_MATCHING`, `HELMUT_AUTH_MODE` u. a.) ist aus der Arbeitsumgebung **nicht lesbar** (Vercel-„sensitive“). Ob die Blob-Daten `sources` / `rawItems` eingefroren sind, ist nur datenseitig, nicht codeseitig belegbar.

4. **Kritische Risiken:** (a) 1,2-MB-Monolith-Blob `helmut_store` läuft wiederkehrend in 10-s-Timeouts → ein Timeout kann einen ganzen Crawl-Save verlieren; (b) Pipeline-Lock ist **fail-open + nicht-atomar** → Doppelverarbeitung/doppelte KI-Kosten möglich; (c) **kein Pipeline-Fehler** landet je in `systemErrors` (kein `lib/helmut` -Modul ruft `recordSystemError`) → Motorfehler sind nur in flüchtigen Logs sichtbar; (d) der Google-News-Degradations-Alarm ist ein **totter Monitor** (liest ein Feld, das `compactStore` beim Speichern entfernt); (e) 247/314 Wissensobjekte ohne Ebene/Feature-Vektor (Altbestand nie gebackfillt) → Relevanz-Matching wirkt nur auf ~21 % des Bestands.
5. **Welche Messungen fehlen:** Crawl-/Pipeline-/Lage-/Understanding-/Briefing-Dauer; Pro-Quelle-„zuletzt erfolgreich“; Understanding-Durchsatz je Lauf (persistiert); Klassifikations-/Embedding-Abdeckung als laufende Kennzahl; KI-Budget-Ausschöpfung als Alarm; relational-vs-Fallback-Quellenpfad je Lauf; Alarm-Zustell-Bestätigung.
6. **Bundestagspilot (überwacht) geeignet? Bedingt ja.** Der fachliche Kern erzeugt nachweislich Vorgänge und Briefings. Für einen *überwachten* Piloten fehlen aber die Beobachtbarkeit (Laufzeiten, Pipeline-Fehler-Persistenz) und zwei Härten (Blob-Timeout-Robustheit, atomarer Lock). Diese sind P0/P1 und klein.
7. **Kern strukturell landtagsfähig? Ja im Kern, nein ohne gezielten Umbau.** Die relationale Quellenschicht, die Geografie-/Ebenen-Modelle (5 Ebenen, `land` / `kommune` real erzeugt), `parliamentTypeOf`, `LANDESPAKET_BY_BUNDESLAND` sind mehr-ebenen-fähig. Berlin/Brandenburg ist aber **dreifach hart gesperrt** und braucht **zwei Codeänderungen** (Landesmodul-Gate entfernen + PARDOK-Live-Modus bauen) plus Datenaktivierung plus Ebenen-Default-Fix. Kein Neubau, aber auch kein reines „Datenflippen“.
8. **Welche Entscheidungen benötigt werden:** siehe §14 (Prod-Env-Werte bestätigen, Backfill freigeben, Blob-Migration terminieren, Landtag-Cutover-Umfang, Scoring scharfschalten, Persistenz von decisions/matching, Monitoring-Zweitkanal).
9. **Wo Auditbericht & Umsetzungsplan liegen:** dieses Dokument + `docs/helmut_datenmotor_umsetzungsplan.md`.

1. Geprüfter Stand (Phase 1 — Audit-Snapshot)

Punkt	Wert	Beleg
Aktueller Branch	<code>claude/helmut-engine-audit-ifkv6r</code>	<code>git branch --show-current</code>
Aktueller Commit (HEAD)	<code>427295c</code> „Querformat-Sperre ... (#94)" (2026-07-16 09:02 +0200)	<code>git log -1</code>
Production-Commit	<code>427295c</code> — Vercel <code>dpl_EaQJBYPwWw75fYUva5J9XtVo4C</code> , target=production, READY, branch <code>main</code>	Vercel <code>list_deployments</code>
Lokal == Production?	Ja. HEAD = Prod-Commit. Working tree sauber (keine lokalen Änderungen).	<code>git status --short</code> (leer)
GitHub <code>main</code> HEAD	<code>427295c</code> (verifiziert)	GitHub <code>list_commits sha=main</code>
⚠ Lokaler <code>origin/main</code> -Ref	<code>fce2e65</code> (2026-07-10, PR #25) — veraltetes Web-Clone-Artefakt , kein Merge-Base zu HEAD, HEAD hat 3 Wurzel-Commits. Nicht die Prod-Linie.	<code>git merge-base origin/main HEAD</code> → keine; <code>git rev-list --max-parents=0 HEAD</code> → 3
Offene Pull Requests	#88 (Draft, Monitoring-Stack inkl. <code>durationMs</code> -Persistenz, Basis = <code>claude/happy-allen-g5qlua</code> , nicht <code>main</code>); #70 (Quellen-Audit-Doku → <code>main</code>); #8 (White-Mode CSS → <code>main</code>)	GitHub <code>list_pull_requests</code>

Punkt	Wert	Beleg
Umgebungen	Vercel (Projekt <code>helmut-pilot</code> , Team „Nohut“, Region <code>fra1</code> , 1 Lambda <code>api/index.js</code> → <code>server.js</code> , <code>maxDuration</code> 300 s); Supabase (Projekt <code>ddckuvvpcytobyfmbvie</code> , <code>eu-west-1</code> , PostgreSQL 17.6, ACTIVE_HEALTHY, RLS auf allen Tabellen aktiv)	<code>vercel.json</code> ; Supabase <code>list_projects</code> / <code>list_tables</code> es
Relevante Datenbanken	Zwei Datenwelten (siehe §3): Legacy-Blob <code>helmut_store</code> (4 Zeilen) + relationale „Quellenarchitektur“ (~38 Tabellen)	Supabase <code>list_tables</code>
Aktive Serverfunktionen	Genau eine Lambda; <code>api/index.js</code> = <code>module.exports = require("../server")</code>	<code>api/index.js</code>
Geplante Prozesse	9 Vercel-Crons (UTC) + 1 geplante GitHub-Action (<code>briefing-watchdog</code> 05:30 UTC)	<code>vercel.json</code> , <code>.github/workflows/</code>
Externe Dienste	Azure OpenAI (<code>gpt-5-mini</code>), Supabase, Google News (Crawl + URL-Decode), Bundestag-DIP-API, CallMeBot-WhatsApp, Monitoring-Webhook, Web-Push (VAPID)	§9, <code>.env.example</code>
Verwendete KI-Modelle	Azure OpenAI <code>gpt-5-mini</code> (685 echte Calls); OpenAI <code>gpt-4.1</code> = Notfall-Fallback, in Prod durch Azure-Vorrang inaktiv	DB <code>llmUsage.model</code> ; <code>ai.js:18-31</code> , <code>.env.example:1-8</code>
Aktive Quellenpakete	7 <code>source_packages</code> (5 aktiv Bund-Basis, 2 <code>prepared</code> : berlin-basis/brandenburg-basis); 163 <code>retrieval_paths</code> (145 Bund + 18 BE/BB inaktiv)	DB; <code>05-quellenarchitektur</code> -Analyse
Aktive Nutzerprofile	3: <code>cem-ince</code> (Pilot), <code>angela-merkel</code> , <code>james-brown</code> (Testpersonen)	DB <code>helmut_store</code> -Keys, <code>briefings</code> , <code>mandate_profiles</code> (2)
Feature-Flags (managed)	<code>HELMUT_UNDERSTANDING_GATE=shadow</code> , <code>HELMUT_PARDOK_DISPATCH=shadow</code> , <code>HELMUT_SOURCE_MODE=on</code>	<code>helmut-flags.json</code> ; Allowlist <code>flags.js:31-35</code>
Unterschied lokal ↔ Production	Keiner auf Code-Ebene (HEAD = Prod). Prod- Env-Werte sind „sensitive“ und nicht einsehbar → als Annahmen markiert (§9).	—

Applizierte DB-Migrationen (13): u. a. `20260714_ko_classification`, `20260717_llm_budget_reservation`, `source_architecture`. **Verifiziert angewendet:** Spalte `knowledge_objects.decision_level` existiert; Funktion `helmut_reserve_llm_call(p_day,p_scope,p_max)` existiert; `llm_budget_counters` hat Zeilen. → Die im Code als „freigabepflichtig“ markierten Migrationen sind **in Production real angewendet** (das entkräftet die Hypothese eines stillen KO-Schreibfehlers wegen fehlender Spalte).

2. Geprüfte / nicht prüfbare Bereiche

Geprüft (belegt): Cron-Verdrahtung & Autorisierung; Crawler (Fetch, RSS/HTML, Google-News-Decode, PARDOK-Streaming); Dedup (2 Mechanismen); Storage-Layer inkl. `compactStore` /Retention/Locks; Understanding (Clustering, KI-Call, Schema, Klassifikation, Feature-Vektor, Gate, Budget); Matching/Decisions/Scoring; Briefing-/Lage-/Radar-Ausgabe; Dokumentlebenszyklus; Fehlerpfade & Monitoring; Landtag-/Cem-Hardcoding; toter Code; Env/Flags/Secrets. Datenseitig: Crawl-Läufe, LLM-Nutzung, Gate-Telemetrie, Wissensobjekte, Rohdokumente, Briefings, Decisions, Matching, Systemfehler.

Nicht prüfbar / ungeklärt (ausdrücklich markiert):

- Prod-Env-Werte** (Vercel „sensitive“): `HELMUT_MAX_LLM_CALLS_PER_DAY` (50 vs. 100?), `HELMUT_V3_MATCHING`, `HELMUT_V3_LAZY_UNDERSTANDING`, `HELMUT_LLM_BUDGET_FAIL_CLOSED`, `HELMUT_UNDERSTANDING_LOCK`,

HELMUT_LLM_RESERVE_UNDERSTANDING , HELMUT_AUTH_MODE , HELMUT_DIP_PRIMARY , HELMUT_MORNING_PUSH_ALL_PROFILES , HELMUT_MONITORING_WEBHOOK_URL . → aus Verhalten/Datenbestand abgeleitet, entsprechend gekennzeichnet.

• **Ephemere Laufzeiten:** Vercel-Runtime-Logs decken nur die letzten 24 h ab; historische Cron-Laufzeiten sind nirgends persistiert → nicht rekonstruierbar (§8).

Helmut Datenmotor — Kritischer Audit

Vertraulich · 2

• **Einfrier-Status der Blob-Daten** (sources =144, rawItems =466): Code liest/schreibt sie weiter — ob die Daten stillstehen, ist datenseitig zu klären, nicht codeseitig.

• **PARDOK-Shadow-Ertrag** auf Production: Dateiablage ist auf Vercel read-only → keine persistente BE/BB-Shadow-Messung.

3. Architektur-Überblick: das duale Datenmodell

Der Motor fährt **zwei parallele, nicht synchron gehaltene Speicher**:

(A) Legacy-JSON-Blob `helmut_store` (4 Zeilen, Supabase):

id	Größe	wichtige Keys
main	1,24 MB	crawlRuns , systemErrors , briefings , politicalItems , sources (144), rawItems (466), lageChecks , pipelineDebugReports
main-p-cem-ince	734 KB	per-Nutzer: briefings , communicationDrafts , politicalItems ...
main-auth	85 KB	llmUsage (1878), pipelineLocks , systemErrors (59), sourceModeShadowLastRun , sessions , users
main-p-james-brown	364 B	quasi leer (Testpersona)

→ Bei **jedem** Schreiben läuft `compactStore` (storage.js:2118) und kappt hart (crawlRuns ≤ 30, zusätzlich `saveCrawlRun` ≤ 20; rawItems ≤ 600; systemErrors ≤ 300; llmUsage ≤ 5000). Der **1,24-MB-Blob** `main` ist die **dokumentierte Timeout-Quelle** (§9).

(B) Relationale „Quellenarchitektur“ (seit Cutover `HELMUT_SOURCE_MODE=on` die aktive Quellenwahrheit):

Gefüllt	Zeilen	Leer (angelegt, ungenutzt)
raw_documents	5733	sources , political_items , llm_usage (!), topic_memory
knowledge_objects	314	interactions , office_outputs , ko_relations
document_findings	1463	electoral_districts , matching_weights
ko_document_links	1230	path_expected_topics , path_expected_entities
gate_shadow_events	4543	personalized_recommendations , daily_tasks
decisions	98	communication_drafts , user_notes , priority_changes
matching_results	38	pipeline_locks (Locks liegen im Blob, nicht hier!)
briefings	13	
retrieval_paths / package_paths	163/165	

Gefüllt	Zeilen	Leer (angelegt, ungenutzt)
<code>geographies</code> / <code>political_entities</code> / <code>publ</code> <code>shers</code>	50/73/6 4	

Helmut Datenmotor — Kritischer AuditVertraulich · 2

Zentrale Wahrheit: Die *Laufzeit-/Betriebsdaten* des Motors (Crawl-Läufe, Kosten, Fehler, Locks) leben im **Blob**; die *Inhaltsdaten* (Dokumente, Wissensobjekte, Fundstellen) leben **relational**. Das erklärt viele Diskrepanzen unten (z. B. „savedItems“ vs. echte neue Dokumente).

4. Vollständiger Prozesskatalog (Phase 3 — Steckbriefe)

Zeiten: Vercel-Crons sind **fix in UTC**. Berlin = UTC+2 (Sommer/CEST, aktuell) bzw. UTC+1 (Winter/CET). Autorisierung aller Crons: `authorizeCron` (server.js:4774) — **fail-closed** (kein `CRON_SECRET` → 503; falsch → 403; Query-Secret nur wenn `HELMUT_ALLOW_QUERY_SECRETS=true`, Default aus).

4.1 `/api/cron/crawl` — Quellen-Crawl (Kernprozess)

- Zweck:** Alle Profil-Quellen abrufen, Rohdokumente speichern, Understanding/Matching/Decisions anstoßen.
- Ausführungsort/Datei/Funktion:** Vercel-Lambda → `server.js:711-714` → `scheduler.js:168 runSourceCrawl(politicianId=cemInceProfile.id)`.
- Auslöser/Startzeit/Zeitzone/Häufigkeit:** Vercel-Cron `0 4 * * *` und `0 20 * * *` (UTC) → **06:00 & 22:00 CEST** (05:00 & 21:00 CET). 2×/Tag.
- Politische Ebene / Mandant:** Bund; **single-tenant** — ohne Login fällt `politicianId` auf `cem-ince` (`server.js:306-308`; nur Admin-Bypass wählt anderes Mandat).
- Tatsächliche Startzeiten (belegt, 20 Läufe 12.–16.07.):** planmäßig 04:03/16:02/20:02 UTC; zusätzlich manuelle Läufe (07:26, 07:43, 08:33, 09:41, 14:35) — Gründer-Trigger während Merge-Sessions.
- Max. Laufzeit:** `/api/cron/crawl` hat **keinen** `withTimeout` → darf bis 300 s (Lambda-Deckel) laufen. `/api/cron/pipeline` (dieselbe Funktion) ist auf 280 s begrenzt.
- Tatsächliche Laufzeit / Ø / Median / min / max:** **NICHT MESSBAR** — keine Dauer wird persistiert (§8). `compactStore` strippt `durationMs` (storage.js:2124-2133), und `saveCrawlRun` berechnet gar kein `durationMs` (scheduler.js:283-309).
- Erfolgsquote / gefundene / neue / doppelte Dokumente (20 Läufe, DB):** 145 Quellen geprüft, **100 % erfolgreich**, **0 Fehler**; `newCandidateItems` **konstant exakt 1012** (= Kandidaten-Cap, kein echter Neuheitswert); `savedItems` 862–939 (**überzeichnet**, siehe §7/§8). Echte Neuheit: nur **55 neue** `raw_documents` in 24 h.
- Folgeprozess:** eager-Understanding (KI), Lazy-Understanding-Vormerkung, Matching-Shadow, Decision-Shadow, source-mode-Shadow-Vergleich — alle **fail-safe** (Fehler verschluckt).
- Wiederholungslogik / Fehlerverhalten:** Kein Retry. Lock-Skip → HTTP 200 „already running“. Shadow-Fehler nur `console.error`, **nie** `recordSystemError`.
- Manueller Neustart:** `GET /api/cron/crawl` mit `Authorization: Bearer <CRON_SECRET>`.
- Überwachung:** keine direkte; nur indirekt über Health-Report-Frische & GitHub-Watchdog (letzterer triggert `pipeline`).
- Status:** aktiv/produktiv, aber **blind** (Fehler nur in flüchtigen Logs).

4.2 `/api/cron/pipeline` — identischer Crawl, zeitbegrenzt

`0 16 * * *` UTC → **18:00 CEST**. `server.js:770-782` → `withTimeout(runSourceCrawl, 280000)`. Loggt `runSourceCrawl ... ms` (ephemer). Ist der Endpoint, den der GitHub-Watchdog auslöst. **De facto der 3. tägliche Crawl.**

4.3 /api/cron/morning-briefing — Morgen-Push

0 5 * * * UTC → 07:00 CEST. server.js:716-768 → buildV3Briefing (0 KI) + Push. **Single-tenant** außer HELMUT_MORNING_PUSH_ALL_PROFILES=1 (Default aus). **Persistiert NICHTS** in die briefings -Tabelle (nur Push + On-Demand-Build) → erklärt, warum dort nur slot=lage -Zeilen liegen. Loggt build=...ms push=...ms (ephemer).
Helmüt Datenmotor — Kritischer Audit Vertraulich · 2

4.4 /api/cron/understanding — dedizierter KI-Verständnis-Lauf

30 5 * * * und 30 21 * * * UTC → 07:30 & 23:30 CEST. server.js:961-973 → runPendingUnderstandingShadow (Budget 240 s). Verarbeitet als pending vorgemerkte Vorgänge. **Ruft das Understanding-Gate NICHT auf** (Gate nur im eager-Pfad). **Gibt immer ok:true zurück, auch bei processed=0** (alle budget-übersprungen) → stille Erfolgsmeldung ohne Arbeit.

4.5 /api/cron/lage-check — Lage-Prüfung

0 10 * * * UTC → 12:00 CEST. server.js:867-884 → setTimeout(runLageCheck, 280000). **Läuft OHNE Pipeline-Lock** (scheduler.js:339). Bei starker neuer Lage: faltet frische Items in Wissensobjekte/Decisions (foldLageItemsIntoV3).

4.6 /api/cron/lage-briefing — Lage-Narrativ-Vorwärmung (einziger KI-Ausgabe-Pfad)

45 5 * * * UTC → 07:45 CEST. server.js:888-914 → buildLageBriefing je Profil (Loop über alle aktiven Profile; deaktivierte übersprungen). Schreibt briefings -Tabelle (slot=lage , Upsert bf-{user}-lage-{Berlin-Tag}). Erklärt die 13 Zeilen (cem 5 / merkel 4 / brown 4).

4.7 /api/cron/health-report — Morgen-Health-Report (einziger aktiver Alarmweg)

0 6 * * * UTC → 08:00 CEST. server.js:814-865 → buildHealthReport + Zustellung via CallMeBot-WhatsApp + optionaler Webhook. ?dryRun=1 baut ohne Versand. **Der Google-News-Achse liegt ein toter Monitor zugrunde** (§9).

4.8 /api/cron/pipeline-status — rein lesender Status (kein Cron-Schedule)

server.js:791-810 . Gibt den letzten Crawl-Lauf zurück; durationMs immer null (Kommentar 803-806 bestätigt: „wird erst persistiert, sobald der Monitoring-Stapel gemergt ist“). Wird vom GitHub-Watchdog nach Client-Timeout gepollt.

4.9 GitHub-Action briefing-watchdog.yml — externer Backstop (triggert echten Crawl!)

- Auslöser:** GitHub-Cron 30 5 * * * UTC → 07:30 CEST + manuell.
- Was er tut:** ruft /api/cron/pipeline (= echter V3-Crawl, nicht nur Health-Check; Kommentar Z.4 „Triggert täglich den V3-Pipeline-Lauf, falls Vercels eigener Cron ausfällt“) und prüft die Antwort; Client-Timeout 330 s, danach pipeline-status -Poll. **GitHub-Failure-Mail** ist der einzige unabhängige E-Mail-Alarm.
- Einschränkung:** prüft nur die Pipeline — nicht Health-Report, nicht Understanding, nicht Kosten.
- Hinweis: health-watch.yml (aus PR #88) **existiert nicht** (unmerged). Alle übrigen .github/workflows/* (staff-backfill x3, pardok-parser, shadow-pilot, sprint9b-verify) sind workflow_dispatch -only (manuell); ci.yml läuft auf Push/PR.

5. Zeitsteuerung (Phase 4)

Ablauf eines Sommertages (UTC → CEST):

UTC	CEST	Prozess	Art
04:00	06:00	crawl	Crawl + Understanding (eager)

UTC	CEST	Prozess	Art
05:00	07:00	morning-briefing	Push (0 KI)
05:30	07:30	understanding + GitHub-watchdog→pipeline	KI-Understanding + 3. Crawl
05:45	07:45	lage-briefing	KI-Narrativ (alle Profile)
06:00	08:00	health-report	Alarm/WhatsApp
10:00	12:00	lage-check	Lage-Refresh
16:00	18:00	pipeline	Crawl (zeitbegrenzt)
20:00	22:00	crawl	Crawl + Understanding (eager)
21:30	23:30	understanding	KI-Understanding

- **Parallelität / Kollisionsrisiko:** Um **05:30 UTC** überlappen `understanding`-Cron und watchdog-getriggerte `pipeline` (die selbst eager-Understanding fährt). Der Pipeline-Lock schützt nur `runSourceCrawl`, **nicht** den Understanding-Pfad (globaler Understanding-Lock ist Default aus). → **Doppelte KI-Understanding-Calls sind zeitlich möglich**; die einzige Bremse ist die Idempotenz (KO existiert bereits).
- **Abhängigkeiten:** morning-briefing (05:00) liest die vom 04:00-Crawl erzeugten Wissensobjekte; lage-briefing (05:45) ebenso. Später eintreffende Dokumente werden erst beim nächsten Crawl/Understanding sichtbar.
- **Später eintreffende Dokumente:** kein Nachtrag in bestehende Briefings; das V3-Briefing wird **bei jedem Read** neu gebaut → automatisch aktuell. Das gecachte Lage-Narrativ wird per `koSetHash` invalidiert.
- **Doppelte Briefings:** ausgeschlossen für Lage (Upsert je `bf-{user}-lage-{Tag}`). Morgen-Briefing wird nicht persistiert.
- **Verpasster Lauf: keine Catch-up-Logik**, kein Tages-Idempotenz-Marker. Vercel holt verpasste Crons nicht nach; der Code auch nicht. Einziger Ersatz: der GitHub-Watchdog triggert die Pipeline unabhängig.
- **Sommer-/Winterzeit:** Da alle Crons **UTC-fix** sind, verschiebt sich **jeder** Prozess im Winter um **1 h früher** in Berliner Ortszeit (z. B. Morgen-Briefing 07:00 → 06:00). Bewusst zu entscheiden, ob das gewünscht ist.
- **Versehentlicher Doppelstart:** möglich über (a) fail-open-Lock bei Storage-Timeout, (b) 05:30-Überlappung, (c) manuelle Trigger ohne Idempotenz-Schutz.

6. Vollständiger Datenfluss (Phase 2) — Weg eines Dokuments

#	Schritt	Datei · Funktion	liest	schreibt	Fehler / stille Fehler	Status
1	Quelle	<code>sources.js</code> , relationaler Plan <code>source-mode.js</code> <code>buildRelationalCrawlPlan</code>	<code>retrieval_paths / packages / package_paths</code> (bei <code>on</code>)	—	Plan leer/ladefehler → still Fallback Alt-Katalog (<code>console.warn</code>)	aktiv (<code>on</code>)
2	Abruf	<code>crawler.js</code> <code>crawlAllSources / crawlSource</code> (RSS/HTML; Google-News-Decode; PARDOK-Streaming)	HTTP	in-memory Items	per-Quelle <code>try/catch</code> → <code>ok:false</code> in <code>crawlRun.errors</code> (max 20)	aktiv
3	Original-Inhalt	nicht gespeichert (DSGVO-Datensparsamkeit: nur <code>summary</code> ≤ 240 Zeichen)	—	—	Volltext bewusst verworfen	by design

#	Schritt	Datei · Funktion	liest	schreibt	Fehler / stille Fehler	Status
4	Textextraktion	<code>crawler.js</code> <code>normalizeRawItems</code> (Regex, kein DOM)	HTML/XML	<code>rawItem</code>	keine „Extraktion fehlgeschlagen“-Zustandsklasse pro Dokument	aktiv
5	Speicherung roh	<code>scheduler.js:221</code> <code>saveRawItems</code> (Blob) + <code>persistRawDocumentsShadow</code> (relational)	Blob-Hashes	<code>helmut_store.rawItems</code> , <code>raw_documents</code> (Upsert <code>rd-<hash></code>)	relationaler Write doppelt fail-safe (<code>scheduler.js:163+224</code>) → still 0	aktiv (<code>V3_STORE</code>)
6	Klassifizierung	<code>understanding.js:501</code> → <code>classification.js</code> <code>classifyKnowledgeObject</code>	KO-Felder	<code>decision_level</code> , <code>political_level</code> (=decision_level), Entitäten, Geografien	<code>try/catch</code> verschluckt → KO ohne Ebene	aktiv (write-time)
7	Wissensojekt	<code>understanding.js:448</code> <code>assembleKnowledgeObject</code> + KI (<code>ai.requestStructuredJson</code> , <code>gpt-5-mini</code> , <code>KNOWLEDGE_OBJECT_SCHEMA</code> , <code>politicianId:null</code> = global)	Cluster	<code>knowledge_objects</code> (Upsert)	Save-Fehler → still skipped-store (KI-Kosten schon ausgegeben)	aktiv
8	Feature-Vektor („embedding“)	<code>understanding.js:522</code> <code>computeFeatureVectorForKnowledgeObject</code> (Token-Hash, kein semantisches Embedding)	KO	<code>embedding</code>	<code>try/catch</code> verschluckt → KO ohne Vektor	aktiv
9	Personen/Themen	in KO-Feldern (<code>mentioned_*</code> , <code>parteien</code> , <code>ausschuesse</code>) + <code>radar.js</code> / <code>radarState.js</code> (2 Engines)	KO	—	2 divergierende Erwähnungs-Engines	aktiv
10	Politische Ebene	<code>decision_level</code> ∈ {bund, land, kommune, eu, international, unknown}	—	KO	Casing: Deriver schreibt klein (<code>bund</code>), Alt-Daten groß (<code>Bund</code>)	aktiv (sparse)
11	Quellenpaket / Mandatsprofil	<code>profile-packages.js</code> <code>resolveProfilePackages</code> (Referenzzählung)	Profil, Pakete	—	Bund-Basis Pflicht für jedes Profil	aktiv
12	Relevanzbewertung	on-read <code>decisions.js</code> <code>decideForUser</code> → <code>matching.js</code> <code>matchProfileToKnowledgeObjects</code> (deterministisch, 0 KI)	Profil + KOs + Feature-Vektoren	— (Read-Pfad); Shadow schreibt <code>matching_results</code> / <code>decisions</code> nur für cem-ince	Shadow-Fehler <code>.catch(()=>null)</code> ohne Log	aktiv
13	Handlungsempfehlung	aus KO-LLM-Feld <code>recommendation</code> / <code>handlungsempfehlung</code>	KO	—	—	aktiv
14	Briefing	<code>server.js:1746</code> <code>buildV3Briefing</code> (0 KI) → <code>briefingContract.js</code>	KOs + on-read Decisions + Quellen	— (nicht persistiert)	Store-Fehler → <code>empty('store-error')</code>	aktiv

#	Schritt	Datei · Funktion	liest	schreibt	Fehler / stille Fehler	Status
1 5	Ausgabe/ Speicherung	Lage-Narrativ (<code>lage.js</code> , 1 KI-Call) → <code>briefings</code> -Tabelle; Home/Radar on-read	KOs	<code>briefings</code> (nur Lage)	Cache-Fehler still verschluckt → KI-Neu- erzeugung	aktiv Vertraulich · 2

Nachvollziehbarkeit (rückwärts): `decisions` / `matching_results` tragen `knowledge_object_id` + `vorgang_id` ;
 KO→Dokument via `ko_document_links` (N:M); Dokument→Quelle via `document_findings` . **Bruchstelle:** Die `briefings` -Tabelle hat **keine** Fremdschlüssel auf `decisions` / `knowledge_objects` (nur `{id,user_id,slot,generated_at,payload}`) — die Kette „warum stand das im Briefing“ liegt nur im opaken `payload` -JSON. `office_outputs` (die einzige Tabelle mit `decision_id` + `knowledge_object_id`) ist **leer**.

7. Dokumentlebenszyklus (Phase 5)

Zustand	Existiert?	entsteht durch	gespeichert wo	Retention / Neuversuch / Löschung
neu	✓	<code>saveRawItems</code> / <code>toRawDocumentRow</code>	Blob <code>rawItems</code> + <code>raw_documents</code>	Blob gekappt auf 600; relational nie gelöscht
gespeichert	✓ (implizit, kein Status-Feld)	Upsert <code>rd-<content_hash></code>	<code>raw_documents</code>	idempotent; keine <code>status</code> -Spalte
zur Verarbeitung vorgesehen (pending)	✓	<code>runLazyUnderstandingShadow</code> → <code>savePendingKnowledgeObject</code>	<code>knowledge_objects</code> . <code>status='pending'</code>	bleibt bis Understanding; Budget-Skip belässt pending → Retry idempotent
in Verarbeitung	✗	—	—	kein <code>processing</code> -Status; nur grobkörniger globaler Lock (Default aus)
erfolgreich verarbeitet	✓	<code>assembleKnowledgeObject</code> (KI ok+valide)	<code>understanding_status='complete'</code>	—
relevant / nicht relevant	✓ (Relation, kein Dok-Flag)	on-read <code>decideForUser</code> ; Shadow <code>runDecision/Matching</code> (nur cem-ince)	<code>decisions</code> / <code>matching_results</code> ; Gate-Telemetrie <code>gate_shadow_events</code>	keine Löschung stale Zeilen
doppelt	✓	<code>dedupeRawDocuments</code> (content_hash) + <code>dedup-global</code> (3 Stufen: canonical / Fingerprint / Domain+Titel-Ähnlichkeit ≥ 0,72 + 2-Tage-Fenster)	Duplikat → nie eigene Zeile, stattdessen <code>document_findings</code> + <code>finding_count++</code>	—
unvollständig	✓	Schema-Validierung schlägt fehl → <code>markUnderstandingFailed</code> ; oder Lese-Qualitätsstatus	<code>understanding_status='failed'</code>	terminal
nicht lesbar	✗ (nur quellenweit)	Fetch-Fehler <code>ok:false</code>	<code>crawlRun.errors</code> (max 20, pro Quelle)	kein Pro-Dokument-Zustand

Zustand	Existiert?	entsteht durch	gespeichert wo	Retention / Neuversuch / Löschung
unbekanntes Format	✗	—	—	<code>document_type</code> zu 99 % null, nie als Ablehnungsgrund
Verarbeitung fehlgeschlagen	✓	LLM-/Schema-Fehler → <code>markFailed</code>	<code>understanding_status</code> bleibt <code>pending</code>	kein Auto-Retry (aus <code>listPendingKnowledgeObjects</code> gefiltert)
erneuter Versuch geplant	✗	—	—	nur manueller <code>resetFailedUnderstanding</code> (kein Cron-Aufrufer)
endgültig fehlgeschlagen	✓ (= failed)	s. o.	<code>knowledge_objects</code>	nie erneut versucht, nie ausgeliefert
archiviert	✗	—	—	kein Archiv/Purge/Retention im Storage (grep = 0)
gelöscht	✗ (für Dokumente/KOs)	nur DSGVO-Profilflöschung (<code>deleteProfileDataV3</code> , nur nutzergebundene Tabellen)	—	<code>raw_documents</code> / <code>knowledge_objects</code> / <code>ko_document_links</code> / <code>document_findings</code> werden NIE gelöscht; Blob nur gekappt

Was mit nicht-für-ein-Briefing-verwendeten Dokumenten passiert: Sie bleiben **dauerhaft** in `raw_documents` / `knowledge_objects` (kein Archiv, keine Löschung, keine Retention). Bei Budget-Engpass bleiben interessante Cluster `pending` und werden idempotent nachgeholt; verstandene, aber nie relevante KOs verbleiben unbegrenzt. **Datenverlust-Schutz:** Upsert-Idempotenz + Dedup-Fundstellen. **Doppelverarbeitungs-Schutz:** Idempotenz + (schwacher) Lock. **Sichtbarkeit der Zustände:** nur Admin-Reads; kein Zustands-Dashboard, keine Alarmer auf Zustandsübergänge.

8. Laufzeitwerte (Phase 3) — vorhanden vs. fehlend

8.1 Vorhandene, belegte Messwerte

Einzige persistierte Laufzeit = pro LLM-Call (`llmUsage.durationMs` , geschrieben in `ai.js` → `recordLlmUsage`). Aggregiert (1878 Einträge, 01.–16.07., Azure `gpt-5-mini`):

callType	n	ok	Ø ms	Median	min	max	Kosten \$	letzte
skipped-understanding-budget	1115	–	–	–	–	–	–	14.07. 20:03
understanding	256	236	7900	8276	24	19278	0,5437	16.07. 07:37
communicationDraft (Büro)	210	210	3880	3647	2106	12963	0,1960	16.07. 07:56
koTagsBackfill (Einmal)	161	161	1609	1466	1121	6481	0,0303	12.07.
skipped-understanding-error	68	–	–	–	–	–	–	15.07. 14:34
v2ScoreAndPrioritize (tot)	26	26	9220	9472	3061	13501	0,0585	06.07.
helmutAssessment (tot)	25	25	4714	4254	2802	14782	0,0387	06.07.
lageBriefing	6	6	5247	5270	3633	6464	0,0087	16.07. 07:39

callType	n	ok	Ø ms	Median	min	max	Kosten \$	letzte
parliamentAssessment	1	1	2965	–	–	–	0,0005	03.07.

Gesamt-LLM-Kosten seit 01.07. ~ \$0,92. Weitere belegte Betriebszahlen: 20 Crawl-Läufe (100 % Quellen-Erfolg, 0 Fehler); `raw_documents` 5733 (55 neu/24 h); Gate-Telemetrie `verstehen 3088 / zurueckstellen 1415 / parken 40`.

8.2 Fehlende Messwerte (müssen im 2. Thread eingebaut werden)

Prozess	Fehlt	Wo einbauen
Crawl / Pipeline	Gesamtdauer (Wall-Clock)	<code>scheduler.js:168</code> <code>t0=Date.now()</code> , <code>durationMs</code> in <code>saveCrawlRun</code> (283) und in die <code>compactStore</code> -Whitelist (<code>storage.js:2124-2133</code>), sonst sofort wieder gestript
Lage-Check	Dauer	<code>scheduler.js:339</code> + <code>saveLageCheck</code> -Payload
Understanding-Batch (eager + Cron)	Loop-Dauer, <code>processed/deferred/reason</code> persistiert , Anzahl <code>skipped-store</code>	eigene Zähler-Zeile im Auth-Store (Muster <code>adminRecoveryLastRun</code>), nicht im großen Blob
Briefing-/Radar-Aufbau	End-to-End-Dauer (0 KI)	<code>server.js:1746</code> / <code>radar.js:234</code> <code>t0</code> / Δ
Pro-Quelle-Gesundheit	<code>last_success_at</code> je Quelle	beim <code>saveRawDocument</code> je <code>source_id</code> fortschreiben → <code>quality-watchdog</code> <code>pathTelemetry=true</code>
Klassifikation/Feature-Vektor	Abdeckungs-Kennzahl (<code>decision_level</code> / <code>embedding</code> null-Quote)	Health-Report-Achse
KI-Budget	Ausschöpfung als Alarm (nicht nur 1115 stille Records)	Health-Report-Achse aus <code>getLlmUsage</code>
Quellenpfad	„relational vs. Fallback“ je Lauf	Flag in <code>crawlRun</code> / <code>pipeline-status</code>

Fazit Laufzeit: Für einen *überwachten* Piloten ist die Nicht-Persistenz jeder Prozessdauer der größte Beobachtbarkeits-Mangel. PR #88 adressiert genau `durationMs`, hängt aber auf Nicht-`main`-Basis fest.

9. Fehler & Überwachung (Phase 6)

Grundmuster: Die Pipeline ist durchgängig „fail-safe“ — aber fail-safe heißt hier fast überall * `console.error` + weiter*, nicht *Systemfehler + Alarm*. `recordSystemError` wird von **KEINEM** `lib/helmut`-Modul aufgerufen (nur `server.js`). Alle Crawl-/KI-/DB-/Dedup-/Matching-Fehler landen ausschließlich in flüchtigen Vercel-Logs.

Fehlerklasse	Erkennung	Protokollierung	Wiederholung	sicherer Endzustand	Warnung/Mensch	Datenverlust-Risiko
Quelle nicht erreichbar	ja (<code>ok:false</code>)	<code>crawlRun.errors</code> (max 20)	nächster Lauf	Lauf läuft weiter	nur bei aggregierter Fehlerrate	gering
leere Antwort	ja (empty feed)	in <code>errors[]</code>	nächster Lauf	ja	nein	gering

Fehlerklasse	Erkennung	Protokollierung	Wiederholung	sicherer Endzustand	Warnung/Mensch	Datenverlust-Risiko
veränderte Seitenstruktur	schwach (Regex-Parser liefert [])	ggf. <code>errors[]</code>	nächster Lauf	ja	nein (schleichend)	mittel (unbemerkte Erosion)
defekte/große PDF	teilw. (PARDOK 64-MiB-Deckel, fail-closed)	<code>errors[]</code>	nächster Lauf	ja	nein	gering
unbekanntes Format	keine Zustandsklasse	—	—	—	nein	gering
Textextraktion fehlgeschlagen	keine Pro-Dok-Klassen	—	—	—	nein	mittel
KI nicht erreichbar (Azure 404)	ja (Exception)	<code>markFailed</code> + <code>skipped-error</code>	kein Auto-Retry	<code>failed</code> (terminal)	nein	hoch (Vorgang dauerhaft unsichtbar)
KI-Timeout	ja	wie oben	kein Auto-Retry	<code>failed</code>	nein	hoch
KI-Ergebnis unbrauchbar	ja (Schema-Validierung)	<code>markFailed</code> + <code>skipped-invalid</code>	kein Auto-Retry	<code>failed</code>	nein	hoch
Datenbankfehler	teilw.	<code>console.error</code> ; Blob-Timeout → 500 (scope api)	kein Retry	evtl. ganzer Lauf verloren	nur als api-Fehler	hoch
Doppelverarbeitung	schwach	—	—	Lock (fail-open)	nein	mittel (Doppelkosten)
fehlendes Mandatsprofil	ja (<code>getActiveProfile</code> neutral)	—	—	neutrale Defaults	nein	gering
falsche Nutzerzuordnung	—	—	—	—	nein	ungeklärt
unvollständiges Briefing	ja (Leerzustände)	Response-only	—	<code>empty('...')</code>	nein	gering
erfolgreicher Lauf ohne neue Daten	nein	—	—	<code>ok:true</code> /200	nein	—
Prozess bleibt ohne Fehler stehen	nein	—	—	—	nein	—
mehrere gleichzeitige Instanzen	schwach (Lock fail-open, TOCTOU)	—	—	—	nein	mittel

Fehlerklasse	Erkennung	Protokollierung	Wiederholung	sicherer Endzustand	Warnung/Mensch	Datenverlust-Risiko
Kostenlimit überschritten	ja (<code>skipped-budget</code>)	1115 <code>llmUsage</code> - Records	idempotent (bleibt pending)	Regel-Fallback	nein (kein Alarm)	gering

Belegte Kernbefunde:

- 1. **Toter Google-News-Monitor:** `buildHealthReport` liest `crawl.googleUrlResolution` (server.js:3201), aber `compactStore` strippt dieses Feld → `gnr` immer `null` → die als SPOF-Frühwarnung beworbene Warnung **feuert nie**.
- 2. **`recordSystemError` -Lücke:** kein Pipeline-Modul meldet je einen Systemfehler → der Motor ist im `systemErrors` - Log unsichtbar.
- 3. **Lock fail-open + TOCTOU:** `acquirePipelineLock` (storage.js:1017-1033) ist ein nicht-atomarer Read-modify-write auf dem Auth-Blob und liefert bei Storage-Fehler `true` → unter Blob-Instabilität Doppelverarbeitung. Der einzige echte Doppel-KI-Schutz (globaler Understanding-Lock) ist Default aus und nirgends verdrahtet.
- 4. **Blob-Timeout = Datenverlustpfad:** 10-s-AbortController (storage.js:1791); der 1,24-MB-Write ohne Retry → bei Timeout während `saveCrawlRun` / `saveRawItems` geht der ganze Lauf verloren. Belegt: wiederkehrend „Supabase storage timed out 10000ms /rest/v1/helmut_store“ (58 api-Fehler, zuletzt 16.07. 05:50).
- 5. **Stille Erfolge:** understanding-Cron `ok:true` bei `processed=0`; crawl-Cron 200 bei Lock-Skip; `newCandidateItems` konstant 1012 → niemand bemerkt einen klemmenden Kandidaten-Cap.
- 6. **quality-watchdog (10 Achsen):** existiert, wird aber nur im Admin-Read mit `signals:{}` instanziiert → alle Frische-Achsen „unbekannt“, **kein Alarmpfad**.
- 7. **Alarmwege:** einziger aktiver = Health-Report 06:00 UTC über CallMeBot-WhatsApp (Gratis-Drittdienst, fail-open/still) + optionaler Webhook. Fällt der Report-Cron aus, merkt es nur der GitHub-Watchdog — der aber nur die Pipeline prüft.
- 8. **Azure-404-Fenster (03.07., 6x):** „DeploymentNotFound“ — Provider-Fehlkonfiguration, inzwischen behoben (Understanding läuft).

10. Datenqualität & Nachvollziehbarkeit (Phase 7)

- **Aktualität:** tagesfrisch (`raw_documents` bis 16.07. 07:34; KOs bis 16.07. 07:37).
- **Vollständigkeit KO-Anreicherung:** **67/314 Wissensobjekte** haben `decision_level` / `political_level` / `embedding` (Feature-Vektor); **247 sind Altbestand vor 14.07. und wurden nie gebackfillt** (verifiziert: von 83 KOs seit 14.07. haben 65 die Ebene). Der Schreibpfad **funktioniert** (Migration angewendet) — es fehlt der Backfill.
- **Ebenen-Verteilung (belegt):** `bund 46, land 8, eu 8, international 3, kommune 2, (null) 247` . → **Multi-Ebenen-Klassifikation funktioniert nachweislich** (auch `land` / `kommune`).
- **„0 x Bund“-Auflösung:** war ein **Casing-Artefakt** — `classification.js` schreibt **klein** (`bund`); Alt-/Debug-Daten nutzen `Bund` . Downstream-Filter auf `Bund` verfehlen neue KOs.
- **Dubletten:** `raw_documents` 5386 distinct `content_hash` von 5733 → 347 kollidieren (per Fundstelle zusammengeführt, keine echten Dubletten-Zeilen).
- **Textextraktion:** Regex-basiert (kein DOM); `summary` ≤ 240 Zeichen (DSGVO). 99 % ohne `document_type` (nie befüllt, aber auch nie als Ablehnungsgrund genutzt).
- **Personen/Organisationen/Themen/Ebene:** in KO-Feldern vorhanden; **2 divergierende Erwähnungs-Engines** (`radar.js` vs. `radarState.js`) mit unterschiedlicher Strenge → Radar-Tab und „Über dich“-Sektion können abweichen.
- **Nutzerzuordnung / Relevanz:** on-read deterministisch für **alle** Profile; die persistierten `decisions` / `matching_results` existieren **nur für cem-ince** und werden im Ausgabepfad **fast nicht gelesen**

(`listDecisions` hat „keinen Produktionsaufrufer“, storage.js:1478).

• **Quellenangabe:** ☒ 93 % direkte Links; KO→Dokument via `ko_document_links`.

• **Rückwärts-Nachvollziehbarkeit bis ... Nutzer:** ☒ Quelle→Dokument→Fundstelle→KO→Decision/Matching. ☒

Bruch bei Briefing→Decision (keine FK-Spalte; nur `payload`-JSON).

Helmut Datenmotor — Kritischer Audit

Vertraulich · 2

11. Ungenutzte Dateien / toter Code (Phase 8)

„Laufzeit-tot“ = kein `require` durch `server.js` / `scheduler.js` / anderen `lib`-Code, nur durch `scripts/`. Nichts wurde gelöscht (reine Lese-Analyse).

Fund	Ort	frühere Rolle	aktuelle Verwendung	Risiko	Empfehlung
<code>staff-backfill.js</code>	<code>lib/helmut/</code>	Stabschef-Felder-Backfill	nur <code>scripts/staff-backfill.js</code> (CLI, dry-run-Default)	niedrig	nach <code>scripts/one-off/</code>
<code>quellenarchitektur/migration-mapper.js</code>	<code>lib/helmut/</code>	Sprint-6-Migration Blob→relational	nur <code>scripts/sprint6-*</code>	niedrig	als abgeschlossene Migration archivieren
<code>quellenarchitektur/cem-shadow-compare.js</code>	<code>lib/helmut/</code>	Cem-Migrations-Shadow-Vergleich	nur <code>scripts/sprint6-*</code>	niedrig	wie oben
<code>quellenarchitektur/understanding-priority.js</code>	<code>lib/helmut/</code>	relevanzbasierte Budget-Auswahl (fertig, getestet, nie scharf)	nur <code>scripts/gate-adversarial-test.js</code>	mittel (verdrängt heute relevante späte Vorgänge, Ankunftsreihenfolge)	scharfschalten (Freigabe) — Kern-Fix gegen Aushungern
<code>ko-classification-backfill.js</code>	<code>lib/helmut/</code>	Altbestand-Ebenen-Nachfüllung	nur <code>scripts/</code> (<code>--execute</code> manuell)	hoch (247 KOs ohne Ebene)	einmalig ausführen / an Cron
<code>generateHelmutAssessment</code> (+Umfeld)	<code>ai.js:350</code>	V2-KI-Bewertung	0 Aufrufer (V3 nutzt deterministisches <code>buildHelmutAssessment</code>)	mittel (versehentliche LLM-Reaktivierung)	als toten V2-Pfad entfernen (nach Bestätigung)
<code>v2ScoreAndPrioritize</code>	—	V2-Scoring	nur Test-Fixture-String	keiner	erledigt
<code>runMorningBriefing</code>	—	V2-Morgen-Briefing	entfernt (Cron ruft V3)	keiner	erledigt
<code>docs/AUDIT_DATENMOTOR_2026-07.md</code>	<code>docs/</code>	Alt-Audit (01.07.)	referenziert nicht existierende Dateien (<code>personalization.js</code> / <code>runtime.js</code> / <code>briefing.js</code>), Single-Tenant-Vor-V3-Framing	mittel (irreführend bei Due Diligence)	als überholt kennzeichnen

Fund	Ort	frühere Rolle	aktuelle Verwendung	Risiko	Empfehlung
docs/V3_MIGRATION_PLAN.md	docs/s/	„Plan“	faktisch abgeschlossenes Cutover-Protokoll	niedrig	umbenennen
Helmut Datenmotor — Kritischer Audit					Vertraulich · 2
tote Env-Flags	.env.example	—	HELMUT_TENANT_JWT_MODE (hart false), SUPABASE_JWT_SECRET / ANON_KEY (nur im stillgelegten JWT-Modus), OpenAI-Pfad (Azure-Vorrang), HELMUT_MONITORING_EMAIL (existierte nie)	niedrig	dokumentieren/entfernen
Falsch-verdächtig (LIVE!)			learning.js (server.js:638/702/708/3046), presentation-backfill.js (server.js:953), quality-watchdog.js (server.js:3726)	—	nicht anfassen

Falle: HELMUT_UNDERSTANDING_GATE=on und HELMUT_PARDOK_DISPATCH=on schalten im aktuellen Code **NICHTS** scharf (der on-Arm ist nicht verdrahtet; pardok fällt bei on still auf off). Wer sie setzt, ändert nichts — **ohne Fehlermeldung**.

Blob-Keys rawItems / politicalItems / topicMemory / sources : Code liest/schreibt sie **weiter** (nicht tot). Ob die *Daten* eingefroren sind, ist datenseitig zu klären (ungeklärt).

Fehlend: kein toter-Code-/require-Graph-Report in CI → solche Module fallen nicht automatisch auf.

12. Landtagsfähigkeit (Phase 9) — Berlin/Brandenburg

12.1 Was strukturell BEREITS mehr-ebenen-fähig ist

- Relationaler Quellenplan (source-mode.js) ist die aktive Wahrheit; geographies mit 5 Ebenen; parliamentTypeOf löst Landtag aus mandate_profiles.politische_ebene auf (config.js:151-164); profileCompleteness kennt „Landtag braucht Bundesland“ (config.js:194); LANDESPAKET_BY_BUNDESLAND = {berlin, brandenburg} (profile-packages.js:28-31); Klassifikation erzeugt real land / kommune ; entity_type -Enum enthält parliament / authority .
- **BE/BB-Seed (20260717) ist in der Live-DB angewendet:** 4 Entitäten, 14 Publisher, 18 Abrufwege, 18 path_expected_levels (alle land), 19 package_paths .

12.2 Die DREIFACH-Sperre (warum heute 0 sichtbare BE/BB-Items)

1. **Hartes Landesmodul-Gate** in buildRelationalCrawlPlan (source-mode.js:46-52,112-116 isLandesmodulPath) schließt **jeden** rp-be- / rp-bb- -Weg aus — auch die Google-News/RSS-Wege, **bevor** Status/Paket geprüft werden. → **CODE-Blocker**.
2. **PARDOK hat keinen Live-Modus:** pardokDispatch gibt in **jedem** Modus items:[] zurück (pardok-dispatch.js:19,72,113); der Live-Pfad ist „bewusst nicht implementiert“. BE/BB-Plenum/Drucksachen können nie in die Pipeline. → **CODE-Blocker**.
3. **Status:** alle 18 BE/BB-Wege needs_review + manual ; Pakete berlin-basis / brandenburg-basis prepared (nicht active). → **DATEN**.

12.3 Ergänzzbarkeit je Element (Data-Flip vs. Codeänderung)

Element	Data oder Code	Beleg
Neue Parlamente	DATA (Entität existiert via Publisher-FK; anreichern)	20260717-Seed Vertraulich · 2
Neue Ausschüsse	DATA (keine Landes-Ausschuss-Entitäten geseedet; <code>classification</code> löst nur Bund auf)	classification.js:127-129
Neue Ministerien	DATA (nur Sammel-Publisher „Ministerien Brandenburg“)	20260717-Seed
Neue Fraktionen	DATA (nur <code>group-agh-linker</code> ; SPD/CDU/AfD/BSW/Grüne fehlen)	20260717-Seed
Neue Behörden	DATA (<code>authority</code> -Enum vorhanden, ungenutzt)	20260713-Migration
Wahlkreise	DATA (<code>electoral_districts</code> komplett leer ; kein Landtagswahlkreis)	DB
Regionalmedien	DATA (Tagesspiegel/rbb24/MAZ geseedet, <code>needs_review</code>)	20260717-Seed
Neue Quellenpakete	DATA (<code>prepared</code> → <code>active</code> flippen)	packages.js:60-70
Eigene Abrufzeiten	CODE (Crons global, kein Land-Scheduler)	vercel.json
Eigene Mandatsprofile	DATA (<code>mandate_profiles</code> -Zeile Landtag; keine BE/BB-Profile existieren)	profile-packages.js:109-114
Landesspezifische Relevanzregeln	CODE (Bund-Bias in <code>classification</code> / <code>scoring</code> / <code>matching</code> ; s. u.)	classification.js:163-174, scoring.js:97-99
Landesspezifische Briefingregeln	CODE (keine Ebenen-/Land-Filterung in <code>lage.js</code> /Briefing)	—

12.4 Bund-Bias, der Landtag heute verhindert (produktweit, nicht nur Demo)

- `neutralProfileDefaults` / `blankProfile` setzen **für jedes Nicht-Cem-Mandat** `politicalLevel='Bund'` , `function='Bundestagsabgeordnete:r'` , `relevantMinistries=['Bundesregierung']` (scheduler.js:1404-1441, server.js:4490-4534, Save-Fallback 4621). → `parliamentTypeOf` klassifiziert ein Mandat **ohne** explizites `politischeEbene='landtag'` als **Bundestag**.
- Crawl-/Scoring-/Relevanz-Heuristiken mit Default `profile=cemInceProfile` feuern für jedes Profil mit fixen Bundesbegriffen: `itemPoliticalWeight` +35 für `bundesregierung` / `bmas` (scheduler.js:1145); `lageCheckSourceWeight` +120 für `bmas|bundesregierung|bundestag|linke|dgb` (427); `hasGovernmentWork` nur Bundesbegriffe (975); föderale Top-Quelle (prio 94) rangiert über Landtag-Quelle (prio 92).
- `matching.js` Synonymkataloge nur Bundestags-Ausschüsse/Bundesparteien (53-74, 118-126). `DIP` (nur Bundestag, WP 21) wird für **jedes** Mandat abgerufen (dip.js, scheduler.js:220) — kein Landtags-Pendant.
- **Sauber gegated (nur Demo):** das volle `cemInceProfile` (Die Linke/BMAS/Niedersachsen) und `demoFallback` -Themen greifen **nur** bei `!authModeOn() && id==='cem-ince'` .

12.5 Urteil Landtag

Der Kern ist strukturell geeignet (relationale Ebenen-Architektur, `land` / `kommune` -Klassifikation, `parliamentTypeOf` , Landespaket-Mapping). **BE/BB ist aber nicht durch reines Datenflippen ergänzbar:** zwingend sind **zwei Codeänderungen** — (1) das harte Landesmodul-Gate entfernen/parametrisieren, (2) den PARDOK-Live-Modus bauen — **plus** Datenaktivierung (Status-Flips, Entitäten, `electoral_districts` , ein Landtags-Mandatsprofil) **plus** der Ebenen-Default-Fix (nicht mehr auto- `Bund`) **plus** der KO-Backfill. Kein Neubau, aber ein umschriebenes Umbaupaket (siehe Umsetzungsplan P2).

13. Risiken (priorisiert)

#	Risiko	Schwere	Beleg
R1	1,24-1MB Monolith Blob → 10-s Timeouts, ganzer Crawl-Save verlierbar ; skaliert mit mehr Mandaten schlechter	kritisch	systemErrors api=58; Vertraulich · 2 storage.js:1791/188-195 (kein Retry)
R2	Pipeline-Lock fail-open + TOCTOU ; Understanding-Lock Default aus → Doppelverarbeitung/Doppel-KI-Kosten, unbemerkt	kritisch	storage.js:1017-1033, 1053-1074
R3	Kein Pipeline-Fehler in <code>systemErrors</code> ; Motor im Fehlerlog unsichtbar	kritisch	grep <code>recordSystemError</code> = 0 in <code>lib/helmut</code>
R4	Toter Google-News-Monitor (liest gestripptes Feld) → SPOF-Frühwarnung feuert nie	hoch	server.js:3201 vs. storage.js:2124-2133
R5	247/314 KOs ohne Ebene/Feature-Vektor → Relevanz-Matching wirkt nur auf ~21 %	hoch	DB; understanding.js:500-525
R6	<code>failed</code> -KOs (88 Skips) nie auto-retry → Vorgänge nach einem KI-Fehlertag dauerhaft unsichtbar	hoch	storage.js:1524; understanding.js:591-599
R7	Keine persistierte Laufzeit → überwachter Pilot ohne Beobachtbarkeit	hoch	§8
R8	<code>savedItems</code> überzeichnet Durchsatz ~15x (Blob-Cap-Artefakt) → falsche Reporting-Basis	mittel	storage.js:2153-2167; DB 55/24h
R9	Ebenen-Casing (<code>bund</code> vs. <code>Bund</code>) → Downstream-Filter verfehlen neue KOs	mittel	classification.js:24 vs. server.js:4972
R10	Alarm nur über CallMeBot (fail-open/still) + optionaler Webhook; kein Ack	mittel	server.js:3267-3279
R11	<code>raw_documents</code> / <code>knowledge_objects</code> ohne Retention/Archiv → unbegrenztes Wachstum	mittel	grep retention=0
R12	Briefing→Decision nicht relational verlinkt → „warum stand das im Briefing“ nicht auditierbar	mittel	storage.js:1630-1636
R13	<code>HELMUT_*_GATE=on</code> / <code>PARDOK=on</code> schalten still nichts scharf (trügerischer Wert)	mittel	understanding.js:685; pardok-dispatch.js:41
R14	05:30-UTC-Überlappung understanding-Cron + watchdog-Pipeline ohne gemeinsamen Lock	mittel	vercel.json + briefing-watchdog.yml
R15	<code>HELMUT_MAX_LLM_CALLS_PER_DAY</code> versehentlich leer → dauerhaft 50 statt 100, nur 1x Warnung	mittel	storage.js:600-611

14. Offene Gründerentscheidungen

Nur Fragen, die **nicht** durch Codeanalyse beantwortbar sind. Detailstruktur (Optionen/Empfehlung/Risiko/Dringlichkeit) siehe `docs/helmut_datenmotor_umsetzungsplan.md`, Abschnitt „Offene Gründerentscheidungen“.

- Prod-Env-Werte bestätigen** (nicht einsehbar): `HELMUT_MAX_LLM_CALLS_PER_DAY` (50 vs. 100?), `HELMUT_V3_MATCHING`, `HELMUT_V3_LAZY_UNDERSTANDING`, `HELMUT_LLM_BUDGET_FAIL_CLOSED`, `HELMUT_UNDERSTANDING_LOCK`, `HELMUT_LLM_RESERVE_UNDERSTANDING`, `HELMUT_AUTH_MODE`, `HELMUT_DIP_PRIMARY`, `HELMUT_MORNING_PUSH_ALL_PROFILES`, `HELMUT_MONITORING_WEBHOOK_URL`.
- KO-Backfill freigeben** (kostenneutral, 0 KI) für die 247 Alt-KOs → Ebene/Feature-Vektor.
- Blob→relational-Migration** der Betriebsdaten (Crawl-Läufe/Locks/Kosten) vor Landtag-Skalierung terminieren.

4. **Landtag-Cutover-Umfang** BE/BB freigeben (2 Codeänderungen + Datenaktivierung) oder verschieben.

5. **Scoring scharfschalten** (`HELMUT_SCORING_MODE`) — heute Default aus; nötig für ebenen-gewichtete Lage.

6. **Persistenz von** `decisions` / `matching_results` : als Output-Quelle nutzen, alle Profile loopen, oder als reine Telemetrie führen + Cleanup.

7. **Monitoring härten:** Pipeline-Fehler in `systemErrors` , `compactStore` -Whitelist um Diagnosefelder, Zweitkanal + Meta-Heartbeat. Vertraulich · 2

8. `sources` / `rawItems` -Blob eingefroren? (Datenfrage, nicht codeseitig belegbar).

9. **Sommer-/Winterzeit-Drift** der Crons akzeptieren oder auf Ortszeit umstellen.

Ende Auditbericht. Umsetzungsschritte mit Priorisierung P0–P3: `docs/helmut_datenmotor_umsetzungsplan.md` .



TEIL II

Umsetzungsplan

Priorisierte Aufgaben (P0 kritisch · P1 vor Bundestagspilot · P2 vor Landtagspilot · P3 später) und die offenen Gründerentscheidungen.

Grundlage: docs/helmut_datenmotor_audit.md (2026-07-16). **Zweck:** priorisierte, umsetzbare Aufgabenliste für den zweiten Thread. In diesem Thread wurde nichts geändert — hier steht, was zu tun ist, warum, wo (Datei:Zeile) und mit welchem Risiko/Aufwand.

Prioritätslegende

- **P0 — kritisch:** Beobachtbarkeit/Sicherheit; ohne diese ist ein überwachter Betrieb blind oder verlustanfällig.
- **P1 — vor Bundestagspilot:** nötig, damit der laufende Cem-/Bundestag-Pilot belastbar und ehrlich ist.
- **P2 — vor Landtagspilot:** nötig, damit Berlin/Brandenburg überhaupt Inhalte liefern.
- **P3 — später:** Hygiene, Skalierung, Aufräumen.

Aufwandslegende: S = klein (<½ Tag), M = mittel (1–2 Tage), L = groß (>2 Tage / Migration / Freigabe).

Reihenfolge-Hinweis: Fast alle P0/P1-Aufgaben sind additive Diagnose-/Härtungs-Änderungen ohne Produktdesign-Eingriff (Lage/Radar/Briefing/Büro/Navigation bleiben unberührt). Migrationen, Zeitplanänderungen, Secret-Rotation und Deployments bleiben Gründer-Freigaben.

P0 — Kritisch (Beobachtbarkeit & Sicherheit)

ID	Aufgabe	Warum	Wo (Datei:Zeile)	Aufwand	Risiko
P0-1	Crawl-/Pipeline-Laufzeit persistieren. t0=Date.now() in runSourceCrawl, durationMs in saveCrawlRun -Objekt UND in die compactStore -crawlRun-Whitelist aufnehmen. Analog Lage-Check/Understanding-Batch.	Ohne persistierte Dauer ist ein überwachter Pilot blind; pipeline-status.duration Ms bleibt sonst null. (Audit §8)	scheduler.js:168, 283-309 ; storage.js:2124-2133 ; lage.js, understanding.js: 731-733	M	S — additiv; ohne Whitelist-Ergänzung wird das Feld sofort wieder gestrippt (Fallstrick beachten)

ID	Aufgabe	Warum	Wo (Datei:Zeile)	Aufwand	Risiko
P 0- 2	compactStore -Whitelist um Diagnosefelder erweitern (durationMs , understanding{} , googleUrlResolution), damit Health-Report/Watchdog echte Daten sehen.	Reanimiert u. a. den toten Google-News-Monitor. (Audit §9-1)	storage.js:2124-2133 vs. server.js:3201	S	S
P 0- 3	Pipeline-Fehler in systemErrors schreiben . Die verschluckten .catch(()=>null) / .catch(()=>{}) in runSourceCrawl / foldLageItemsIntoV3 durch einen recordSystemError -fähigen Sammler ersetzen (fail-safe, nur Metadaten).	Kein lib/helmut -Modul meldet je einen Systemfehler → der Motor ist im Fehlerlog unsichtbar. (Audit §9-2, R3)	scheduler.js:224, 244,263,267 ; accounts.js:594 recordSystemError	M	S — nur additives Logging
P 0- 4	Atomaren Pipeline-Lock . acquirePipelineLock von nicht-atomarem Blob-Read-modify-write auf einen DB-Advisory-Lock / atomaren Upsert (analog helmut_reserve_llm_call) umstellen; fail-open → fail-closed bei Storage-Fehler; globalen Understanding-Lock aktivieren.	Verhindert Doppelverarbeitung/ Doppel-KI-Kosten (bes. 05:30-Überlappung). (Audit §9-3, R2, R14)	storage.js:1017-1033, 1053-1074 ; understanding.js:537-538	M	M — Verhaltensänderung am Lock; sorgfältig testen
P 0- 5	Blob-Timeout-Robustheit . writeSupabaseStore / readSupabaseStore mit Retry+Backoff und/oder Verkleinerung des main -Blobs (Crawl-Läufe/Locks aus dem Blob ziehen). Sofort-Minimallösung: Retention crawlRuns senken + Locks relational.	Ein 10-s-Timeout verliert heute einen ganzen Crawl-Save (kein Retry, kein Teil-Commit). (Audit §9-4, R1)	storage.js:141-158,188-195,1791-1814	L	M — berührt zentralen Speicherpfad

P1 — Vor Bundestagspilot (Ehrlichkeit & Datenqualität)

ID	Aufgabe	Warum	Wo	Aufwand	Risiko
P1 -1	KO-Klassifikations-Backfill ausführen (ko-classification-backfill.js , 0 KI) für die 247 Alt-KOs → decision_level / political_level /Feature-Vektor.	Relevanz-Matching wirkt heute nur auf ~21 % des KO-Bestands. (Audit §10, R5)	lib/helmut/ko-classification-backfill.js ; scripts/ko-classification-backfill.js -execute	S	S — idempotent, kostenneutral; Gründer-Freigabe für Prod-Write
P1 -2	Ebenen-Casing vereinheitlichen (bund klein als Kanon) + Downstream-Filter/Debug-Seed angleichen.	Filter auf Bund (groß) verfehlen neue KOs. (Audit §10, R9)	classification.js:24 ; server.js:4972 ; alle political_level -Vergleiche	S	S

ID	Aufgabe	Warum	Wo	Aufwand	Risiko
P1 -3	Understanding-Priorisierung scharfschalten (<code>understanding-priority.js</code> verdrahten) statt Ankunftsreihenfolge.	Bei Budgetdeckel (1115 Skips) entscheidet heute Zufall, welche Vorgänge verstanden werden; späte amtliche Vorgänge hungern aus. (Audit §11)	<code>understanding.js:721-730</code> ; <code>quellenarchitektur/understanding-priority.js</code>	M	M — Freigabeentscheidung
P1 -4	failed -KO-Recovery an einen Cron/Recovery-Lauf hängen (begrenzter Auto-Retry mit Zähler) statt manuellem <code>resetFailedUnderstanding</code> .	88 Skips/Fehler-KOs sind heute dauerhaft unsichtbar. (Audit §7, R6)	<code>storage.js:1583-1585</code> ; <code>understanding.js:591-599</code>	M	S
P1 -5	Durchsatz ehrlich melden: <code>savedItems</code> (Blob-Cap-Artefakt, ~15x überzeichnet) durch echte neue <code>raw_documents</code> -Deltas ersetzen; <code>newCandidateItems=1012</code> -Klemme untersuchen.	Reporting-Basis lügt heute über den Durchsatz. (Audit §8, R8)	<code>scheduler.js:280-298</code> ; <code>storage.js:2153-2167</code> ; <code>crawler.js:43-50</code>	M	S
P1 -6	KI-Budget- & „Erfolg-ohne-Arbeit“-Achse im Health-Report. Budget-Ausschöpfung (<code>skipped-*</code> -Rate) und <code>processed=0</code> -Läufe in <code>report.ok</code> einfließen lassen; Pro-Quelle-„zuletzt erfolgreich“.	Heute alarmiert nichts bei Budget-Erschöpfung oder stillem Leerlauf. (Audit §9-5/-7)	<code>server.js:buildHealthReport~3098-3232</code> ; <code>quality-watchdog.js:502</code>	M	S
P1 -7	Monitoring-Zweitkanal + Meta-Heartbeat. Sicherstellen, dass ein unabhängiger Alarmweg existiert (Webhook), und den GitHub-Watchdog um einen <code>health-report?dryRun=1</code> -Check erweitern.	Einziger Alarm (CallMeBot) ist fail-open/still; fällt der Report-Cron aus, merkt es niemand. (Audit §9-7, R10)	<code>server.js:3240-3279</code> ; <code>.github/workflows/briefing-watchdog.yml</code>	M	S
P1 -8	Radar-Störungswahrheit. <code>buildRadarForUser</code> auf <code>listKnowledgeObjects({signalError:true})</code> umstellen (wie Lage), damit Store-Ausfall nicht als „ruhig/keine Erwähnung“ getarnt wird.	Radar tarnt heute DB-Fehler als leeres Ergebnis. (Audit §9, Lage-Radar-Analyse)	<code>radar.js:246</code> vs. <code>lage.js:301</code>	S	S — keine Designänderung, nur Ehrlichkeit
P1 -9	Stale-Kommentare korrigieren: <code>source-mode.js:14-15</code> behauptet fälschlich „on nicht aktiviert“ (ist live); <code>classification.js:159-160</code> „sonst Bund“ (Code gibt <code>unknown</code>); <code>docs/AUDIT_DATENMOTOR_2026-07.md</code> als überholt kennzeichnen.	Widersprüche zwischen Code-Kommentar und Realität führen bei Due Diligence/Betrieb in die Irre. (Audit §11)	genannte Stellen	S	keiner

P2 — Vor Landtagspilot (Berlin/Brandenburg)

ID	Aufgabe	Warum	Wo	Aufwand	Risiko
P 2- 1	Hartes Landesmodul-Gate entfernen/parametrisieren in <code>buildRelationalCrawlPlan</code> , sodass aktivierte BE/BB-Wege durchlaufen (Google-News/RSS zuerst, PARDOK separat).	Ohne diesen Code-Fix wirkt kein Datenflip; alle <code>rp-be-</code> / <code>rp-bb-</code> -Wege werden vorab ausgeschlossen. (Audit §12.2-1)	<code>source-mode.js:35-52,112-116</code>	M	M — muss hinter Freigabe/Flag

ID	Aufgabe	Warum	Wo	Aufwand	Risiko
P 2- 2	PARDOK-Live-Modus bauen (structured_download → <code>raw_documents</code>) für BE/BB-Plenum/Drucksachen. Helmut Datenmotor — Kritischer Audit	Heute liefert <code>pardokDispatch</code> in jedem Modus 0 Items; Live ist „bewusst nicht implementiert“. (Audit §12.2-2)	<code>quellenarchi</code> <code>tektur/pardok</code> - <code>dispatch.js:1</code> <code>6-19,39-</code> <code>42,72-113</code> ; Parser vorhanden <code>pardok-</code> <code>parser.js</code>	L	M — neuer aktiver Ingest-Pfad: Byte-Deckel bereits vorhanden
P 2- 3	Ebenen-Default entkoppeln. <code>blankProfile</code> / <code>neutralProfileDefaults</code> / Save-Fallback dürfen <code>politicalLevel</code> nicht mehr auto- <code>Bund</code> setzen; aus <code>mandate_profiles.politische_ebene</code> ableiten.	<code>parliamentType0</code> <code>f</code> klassifiziert sonst jedes Landtagsmandat als Bundestag. (Audit §12.4, R Landtag)	<code>scheduler.js</code> <code>:1404-1441</code> ; <code>server.js:44</code> <code>90-4534,4621</code> ; <code>config.js:16</code> <code>2-164</code>	M	M — berührt Profil- Defaults produktweit
P 2- 4	BE/BB-Daten aktivieren: Pakete <code>berlin-</code> <code>basis</code> / <code>brandenburg-basis</code> <code>prepared→active</code> , Wege <code>needs_review→healthy</code> + <code>manual→auto</code> ; fehlende Entitäten (Landes-Ausschüsse, Fraktionen SPD/CDU/AfD/BSW/Grüne, Behörden) seeden; <code>electoral_districts</code> (Landtagswahlkreise) füllen; je ein BE- und BB- Mandatsprofil anlegen.	Datenseitige Voraussetzung, damit der Plan überhaupt BE/BB- Quellen enthält. (Audit §12.3)	Seeds <code>20260717_*</code> ; <code>packages.js:</code> <code>60-70</code> ; <code>mandate_prof</code> <code>iles</code>	L	M — reine Daten; Gründer- Freigabe für Prod-Seed
P 2- 5	Landes-Relevanz-/Scoring-Kataloge. <code>matching.js</code> -Synonyme + <code>itemPoliticalWeight</code> / <code>hasGovernmentWor</code> <code>k</code> / <code>lageCheckSourceWeight</code> um Landtag/Landesregierung/Staatskanzlei/Landesmi- nisterium ergänzen; <code>LEVEL_IMPORTANCE</code> pro Mandat konfigurierbar.	Sonst wird Landes- Regierungsarbeit nicht als relevant erkannt und Bund dominiert die Lage. (Audit §12.4, Output- Analyse)	<code>matching.js:</code> <code>53-74,118-</code> <code>126</code> ; <code>scheduler.js</code> <code>:427,975,114</code> <code>5</code> ; <code>scoring.js:9</code> <code>7-99</code>	M	M
P 2- 6	Landtags-Primärquelle (Pendant zu DIP) oder BE/BB-Parlamentsdokumentation prüfen/anbinden.	DIP ist Bundestag- only; Landtagsmandate hätten keine amtliche Primärquelle. (Audit §12.4)	<code>dip.js</code> ; PARDOK-Wege	L	M
P 2- 7	Scoring scharfschalten (<code>HELMUT_SCORING_MODE</code>) mit landtauglichen <code>LEVEL_IMPORTANCE</code> -Gewichten, damit ebenen- gewichtete Lage + differenzierte Leerzustände aktiv sind.	Heute Default aus → Recency-Fallback, Bund-lastig für alle. (Audit §10, Lage- Analyse)	<code>scoring.js:3</code> <code>2-38</code> ; <code>lage.js:316</code>	M	M — Gründerents- cheidung

P3 — Später (Hygiene & Skalierung)

ID	Aufgabe	Warum	Wo	Aufwand
P3 -1	Retention/Archiv für <code>raw_documents</code> / <code>knowledge_objects</code> (unbegrenztes Wachstum).	R11	Storage	M

ID	Aufgabe	Warum	Wo	Aufwand
P3 -2	Briefing→Decision relational verlinken (<code>decision_ids</code> / <code>ko_ids</code> in <code>briefings</code>) für revisionssichere Nachvollziehbarkeit. Helmut Datenmotor — Kritischer Audit	R12, Audit \$6	<code>storage.js:1630-1636</code>	M Vertraulich · 2
P3 -3	Toten V2-KI-Pfad entfernen (<code>generateHelmutAssessment</code> + Umfeld in <code>ai.js</code>) nach Bestätigung, dass <code>buildHelmutAssessment</code> endgültig ist.	Audit \$11	<code>ai.js:350-420,1201</code>	S
P3 -4	Einmal-/Migrations-Module archivieren (<code>staff-backfill</code> , <code>migration-mapper</code> , <code>cem-shadow-compare</code> , <code>ko-classification-backfill</code>) nach <code>scripts/one-off/</code> .	Audit \$11	<code>lib/helmut/</code>	S
P3 -5	require-Graph-/Dead-Code-Scan in CI (<code>scripts/deadcode-scan.js</code>).	Audit \$11	CI	S
P3 -6	Zwei Erwähnungs-Engines konsolidieren (<code>radar.js</code> vs. <code>radarState.js</code>).	Audit \$10	beide	M
P3 -7	<code>decisions</code> / <code>matching_results</code> bereinigen (verwaiste Zeilen; heute nie gelöscht) oder als Output-Quelle nutzen.	Audit \$10, Output-Analyse	<code>storage.js:1443-1473</code>	M
P3 -8	Cron-Zeitzone: entscheiden, ob DST-Drift (UTC-fix) akzeptiert oder auf Ortszeit umgestellt wird.	Audit \$5	<code>vercel.json</code>	S
P3 -9	Boot-Zeit-Env-Selbstcheck (nur „gesetzt/nicht gesetzt“, nie Werte) in den Health-Report.	Audit \$2, Env-Analyse	Server-Boot	S
P3 -10	<code>document_type</code> -Befüllung (99 % null) falls für Filter/Anzeige gewünscht.	Audit \$10	Understanding/Crawler	M

Zuordnung: Prozesse ohne ausreichende Laufzeitmessung → wo Messung einbauen

Prozess	Messung fehlt	Einbauort (2. Thread)	Priorität
Crawl / Pipeline	Gesamtdauer	<code>scheduler.js:283</code> + <code>compactStore</code> -Whitelist	P0-1
Lage-Check	Dauer	<code>scheduler.js:339</code> + <code>saveLageCheck</code>	P0-1
Understanding-Batch (eager+Cron)	Loop-Dauer, <code>processed/deferred</code> persistiert, <code>skipped-store</code>	Auth-Store-Zähler-Zeile	P0-1/P1-6
Briefing-Aufbau (<code>buildV3Briefing</code>)	End-to-End-Dauer	<code>server.js:1746</code>	P1-6
Radar-Aufbau	Dauer (keine Telemetrie)	<code>radar.js:234</code>	P3
Pro-Quelle	<code>last_success_at</code>	<code>saveRawDocument</code> / <code>retrieval_paths</code>	P1-6
Quellenpfad	relational vs. Fallback je Lauf	<code>scheduler.js:503-514</code>	P1-5
KI-Budget	Ausschöpfung als Alarm	Health-Report	P1-6

Offene Gründerentscheidungen

Nur Fragen, die die Codeanalyse nicht beantworten kann.

Helmut Datenmotor — Kritischer Audit

Vertraulich · 2

E1 — Prod-Env-Werte bestätigen (nicht einsehbar)

1. **Entscheidung:** Die tatsächlichen Vercel-Env-Werte offenlegen/bestätigen.
2. **Einfache Erklärung:** Der Code liest „sensitive“ Variablen nicht; wir kennen nur die Datei-Flags + abgeleitetes Verhalten.
3. **Warum notwendig:** Budgetdeckel, Fail-Closed-Verhalten, Locks, Matching-Aktivierung hängen daran.
4. **Optionen:** (a) Werte als sicheres Boolean-Inventar teilen; (b) Boot-Selbstcheck einbauen (P3-9); (c) unverändert lassen.
5. **Empfehlung:** (a)+(b).
6. **Vorteil:** Ende der Annahmen; belastbare Betriebsbasis.
7. **Risiko:** minimal (nur „gesetzt/nicht gesetzt“, keine Werte in Logs).
8. **Folge ohne Entscheidung:** zentrale Betriebsparameter bleiben ungeklärt (u. a. 50 vs. 100 Calls/Tag).
9. **Dringlichkeit:** hoch.
10. **Betroffen:** LLM-Budget, Auth-Modus, Matching, Monitoring.

E2 — KO-Klassifikations-Backfill freigeben

1. **Entscheidung:** `ko-classification-backfill` einmalig auf Prod ausführen.
2. **Erklärung:** 247 alte Wissensobjekte haben keine politische Ebene und keinen Feature-Vektor.
3. **Warum notwendig:** Ohne Ebene/Vektor findet das Relevanz-Matching sie nicht.
4. **Optionen:** (a) einmal ausführen; (b) zusätzlich an einen Cron hängen; (c) lassen.
5. **Empfehlung:** (a) sofort, (b) mittelfristig.
6. **Vorteil:** ~3× größerer nutzbarer KO-Bestand, kostenneutral (0 KI).
7. **Risiko:** sehr gering (idempotent, additiv).
8. **Folge ohne Entscheidung:** Matching bleibt auf ~21 % beschränkt; Landtag-Ebenen-Trennung unmöglich.
9. **Dringlichkeit:** hoch.
10. **Betroffen:** Matching, Lage, Radar, Landtagsfähigkeit.

E3 — Blob→relational-Migration der Betriebsdaten terminieren

1. **Entscheidung:** Crawl-Läufe/Locks/Kosten aus dem 1,24-MB-Blob in relationale Tabellen ziehen.
2. **Erklärung:** Der Monolith-Blob läuft wiederkehrend in 10-s-Timeouts; ein Timeout kann einen Crawl-Save verlieren.
3. **Warum notwendig:** Der Blob wächst mit jedem Mandanten → skaliert nicht für Landtag/Mehrkunden.
4. **Optionen:** (a) sofortige Minimallösung (Retention senken + Locks relational, P0-5); (b) volle Migration (L); (c) lassen.
5. **Empfehlung:** (a) vor Bundestagspilot, (b) vor Landtag/Skalierung.
6. **Vorteil:** kein Lauf-Verlust; robustere Beobachtbarkeit.
7. **Risiko:** mittel (zentraler Speicherpfad; sorgfältig testen).
8. **Folge ohne Entscheidung:** sporadischer Datenverlust bleibt, verschärft sich mit Wachstum.
9. **Dringlichkeit:** hoch (P0 für Minimallösung).
10. **Betroffen:** gesamter Motor.

E4 — Landtag-Cutover Berlin/Brandenburg

1. **Entscheidung:** BE/BB scharfschalten (2 Codeänderungen + Datenaktivierung) oder verschieben.
2. **Erklärung:** BE/BB ist heute dreifach hart gesperrt und liefert 0 Items.
3. **Warum notwendig:** Ohne Umbau kein Landtagsinhalt — kein reines Datenflippen.

4. **Optionen:** (a) voller Cutover (P2-1..P2-7); (b) nur Google-News/RSS-Landeswege (ohne PARDOK-Plenum); (c) verschieben.
5. **Empfehlung:** (b) als Zwischenschritt (schnell, ohne PARDOK-Code), dann (a).
6. **Vorteil:** frühe BE/BB-Abdeckung über Medien-/Fraktionswege.
7. **Risiko:** mittel; hinter Flag/Freigabe, Bundestagsbetrieb unberührt.
8. **Folge ohne Entscheidung:** Landtag bleibt strukturell vorbereitet, aber inaktiv.
9. **Dringlichkeit:** mittel (nach Bundestagspilot-Härtung).
10. **Betroffen:** Quellenarchitektur, Klassifikation, Scoring, Profile.

E5 — Scoring scharfschalten (`HELMUT_SCORING_MODE`)

1. **Entscheidung:** die 3-Dimensionen-Bewertung (Wichtigkeit/Relevanz/Handlungsfähigkeit) aktivieren.
2. **Erklärung:** Heute aus → Lage/Radar nutzen reines Ähnlichkeits-Matching + Recency-Fallback.
3. **Warum notwendig:** ebenen-gewichtete Lage (Bund vs. Land) und ehrliche Leerzustände (Datenlücke vs. ruhig).
4. **Optionen:** (a) an, mit Bund-Gewichten; (b) an, mit landtauglichen Gewichten; (c) aus lassen.
5. **Empfehlung:** (a) für Bundestag jetzt, (b) für Landtag.
6. **Vorteil:** differenziertere, ehrlichere Priorisierung.
7. **Risiko:** mittel (verändert sichtbare Rangfolge — vor Aktivierung gegen Realdaten prüfen).
8. **Folge ohne Entscheidung:** Lage bleibt recency-getrieben, Landtag strukturell Bund-lastig.
9. **Dringlichkeit:** mittel.
10. **Betroffen:** Lage, Radar, Helmut-Stand.

E6 — Rolle von `decisions` / `matching_results`

1. **Entscheidung:** persistierte Relevanz-Tabellen als Output-Quelle nutzen, über alle Profile schreiben, oder als Telemetrie führen.
2. **Erklärung:** Heute werden sie fast nie gelesen (Output = on-read-Recompute), nur für `cem-ince` geschrieben, nie bereinigt.
3. **Warum notwendig:** Klärt, ob Persistenz gebraucht wird und ob weitere Mandate sie brauchen.
4. **Optionen:** (a) als Telemetrie führen + Cleanup; (b) Read-Pfad umstellen + alle Profile loopen; (c) ganz entfernen.
5. **Empfehlung:** (a) kurzfristig; (b) wenn Persistenz-Vorteile (Historie) gewünscht.
6. **Vorteil:** weniger Verwirrung, kein verwaister Datenberg.
7. **Risiko:** gering.
8. **Folge ohne Entscheidung:** Schreib-Last ohne Nutzen; falsche Interpretation „was der Nutzer sieht“.
9. **Dringlichkeit:** niedrig-mittel.
10. **Betroffen:** Ausgabe, Storage, Landtag-Skalierung.

E7 — Sind `sources` / `rawItems` -Blob-Daten eingefroren?

1. **Entscheidung:** klären, ob die Blob-Kataloge (`sources` =144, `rawItems` =466) noch aktiv gepflegt werden.
2. **Erklärung:** Der Code liest/schreibt sie weiter; ob die *Daten* stillstehen, ist codeseitig nicht belegbar.
3. **Warum notwendig:** Betrifft, ob der alte Katalog-Pfad noch Wahrheit oder Altlast ist.
4. **Optionen:** (a) datenseitig prüfen (Zeitstempel); (b) als Fallback belassen; (c) abschalten.
5. **Empfehlung:** (a), dann entscheiden.
6. **Vorteil:** Klarheit über Alt-Pfad; ggf. kleinerer Blob (Timeout-Entlastung).
7. **Risiko:** gering.
8. **Folge ohne Entscheidung:** unklarer Alt-Pfad, größerer Blob.
9. **Dringlichkeit:** niedrig.
10. **Betroffen:** Storage, Quellenpfad.

Erstellt aus `docs/helmut_datenmotor_audit.md`. Alle Datei:Zeile-Verweise beziehen sich auf HEAD = Production-Commit `427295c`.